# MathRAG: A Multimodal Retrieval-Augmented Generation System for Mathematical Question Answering

Anonymous Author(s)

## Abstract

## CCS Concepts

• **Computing methodologies** → **Information extraction**; *Learning latent representations*; • **Information systems** → *Question answering*.

## Keywords

mathematical information retrieval, multimodal retrieval, retrieval-augmented generation, formula retrieval, image retrieval, question answering

**ACM Reference Format:**

Anonymous Author(s). 2026. MathRAG: A Multimodal Retrieval-Augmented Generation System for Mathematical Question Answering. In *Proceedings of Conference Title (Conference Acronym '26).* ACM, New York, NY, USA, 6 pages. https://doi.org/TBD

## 1 Introduction

Mathematical question and answering is a difficult task, often presented to standard retrieval systems. Many queries are written in natural language. However, most relevant supporting information may be found in formulas or figures instead of just the text. As a result, a system that retrieves only textual semantics of a query can miss part of the source material.

MathRAG addresses this problem with a modular retrieval-augmented generation pipeline that separates retrieval by modality and unifies evidence only at the generation stage. The implementation has three retrieval branches. The text branch searches dense OpenSearch indices over mathematical discussion posts. The formula branch retrieves symbolic matches through a graph-based pipeline that combines dense, sparse, and structural signals. The image branch retrieves visually relevant mathematical figures from a dedicated image dataset using a vision-language model and FAISS. An orchestrator executes these branches concurrently, converts their outputs into labeled text evidence, and forwards the resulting prompt to a local language model.

The system does not combine all retrieved mathematical content into a single output. Instead, it preserves modality-specific retrieval mechanisms that are better suited to the data they search, then joins their outputs after post processing into a single answer-generation step. The result is a multimodal RAG pipeline in which retrieval is specialized and generation is centralized.



*Conference Acronym '26, City, Country*



ACM ISBN TBD
https://doi.org/TBD

The goal of this paper is to present each retrieval branch and explain how the orchestrator module utilizes each retrieval system to output an answer to a given query.

## 2 Related Work

### 2.1 Mathematical Information Retrieval

Mathematical information retrieval differs from standard text retrieval because relevance is often expressed through notation as much as through language. Systems such as Tangent-S and Tangent-CFT showed that mathematical structure can be retrieved effectively when formulas are represented as trees or structural fragments rather than as flat strings. More recent work has also explored dense neural encoders for mathematical text and formulas, including transformer-based models adapted to technical sources. These lines of work motivate a system that treats text and formulas as related but distinct retrieval problems.

EXPAND THIS SECTION

### 2.2 Multimodal RAG

Retrieval-augmented generation is especially attractive for mathematical question answering because the evidence needed to answer a query is often spread across several representations. In practice, however, multimodal RAG systems must decide whether to learn a single shared retrieval space for all evidence or to retain modality-specific retrieval pipelines and combine them later. MathRAG follows the second strategy. Its text, formula, and image branches are retrieved independently, and only the resulting evidence is fused at prompt construction. This makes the system easier to extend and lets each modality use a retrieval method that matches its structure.

## 3 MathRAG System

MathRAG is implemented as a retrieval-augmented generation pipeline for mathematical question answering. Given a user query, the orchestrator dispatches three retrieval jobs, one for text, one for formulas, and one for images. Each branch returns evidence in its own native form, but the orchestrator converts those outputs into labeled text blocks before generation. The final answer is then produced by a local language model operating over the combined prompt.

Let $R_{\text{text}}(q)$, $R_{\text{formula}}(q)$, and $R_{\text{image}}(q)$ denote the evidence returned by the three retrieval branches for query $q$. The orchestrator combines these outputs as:

$$\mathcal{R}(q) = R_{\text{text}}(q) \cup R_{\text{formula}}(q) \cup R_{\text{image}}(q) \tag{1}$$

and forwards the labeled evidence set $\mathcal{R}(q)$ to the prompt construction stage.

The remainder of this section describes the three retrieval branches and the final generation step in the order used by the live system.

## 3.1 Text Search

The text retrieval branch answers natural language queries over a large dataset of mathematical text, retrieving document hits from five OpenSearch indices covering Math Stack Exchange, MathOverflow, Mathematica, Wikipedia, and YouTube. Documents are indexed with pre-computed dense vector fields, and retrieval is performed by $k$-nearest-neighbor search over those embeddings.

*3.1.1 Dataset and Embedding Architecture.* Documents are indexed into OpenSearch with a stored vector field computed by a fine-tuned sentence transformer. The deployed service loads a local checkpoint built from `all-mpnet-base-v2` and serves 768-dimensional dense embeddings. Both the embedding model and the OpenSearch client are initialized on first use and retained in memory across requests, which avoids repeated model loading and connection setup during interactive querying.

*3.1.2 Query Preprocessing.* Before encoding, the raw query $q$ is normalized through three sequential steps. First, the text is lowercased and leading or trailing whitespace is removed. Second, any remaining HTML markup is stripped using BeautifulSoup with the `lxml` parser. Third, LaTeX dollar-sign delimiters are removed.

*3.1.3 Dense Retrieval.* The preprocessed query is passed through the sentence-transformer encoder to obtain a dense query vector. Each indexed document is represented by a stored vector, and similarity is computed with the OpenSearch knn query on the `body_vector` field. A single request is issued over all five indices, the returned hits are ranked by score, and the top-$K$ results are forwarded to the orchestrator. The service defaults to $K = 200$.

*3.1.4 Context Enrichment via URL Scraping.* For each result whose document identifier is an HTTP URL, the text handler scrapes the source page to obtain fuller context than the stored index field alone provides. The scraper fetches the page with a descriptive User-Agent header, parses the HTML with BeautifulSoup, and extracts the question body together with up to two answer bodies from the Stack Exchange `.s-prose` blocks. If scraping fails for any reason, the stored body text from the OpenSearch index is used as a fallback.

## 3.2 Formula Search

*3.2.1 Related Work and Paradigms in MathIR.* Mathematical Information Retrieval (MathIR) presents a fundamentally different challenge than standard Natural Language Processing (NLP). Mathematical notation is compact, structured, and follows strict rules. Small changes to a formula, such as shifting a variable from a baseline to a superscript like ($x^2$ vs. $x_2$), can completely change its meaning and how it is interpreted. Historically, MathIR systems have attempted to solve this using either flattened structural encodings or dense transformer-based embeddings.

*Tree-Based and N-Gram Structural Encodings.* Early high-precision MathIR systems abandoned raw string matching in favor of tree-based representations. Systems like Tangent-S [**?** ] pioneered the use of Symbol Layout Trees (SLTs) and Operator Trees (OPTs) to capture both the visual and semantic hierarchies of MathML. Tangent-S operates by extracting maximum subtree isomorphisms between a query and a document. While this provides exceptional exact-match precision, rigorous subtree matching is computationally exhaustive, making real-time retrieval over millions of documents infeasible without aggressive pre-filtering.

To address this latency, subsequent models like Tangent-CFT [**?** ] introduced structural N-gram embeddings. Tangent-CFT extracts localized, depth-first paths (e.g., parent-child-sibling tuples) from the SLT and OPT, and embeds these paths into a continuous vector space using unsupervised fastText models. By representing a formula as a bag-of-structural-words, Tangent-CFT allows for fast inner-product similarity search. Limitations: While computationally lightweight, N-gram extraction inherently flattens the formula's topology. By shredding the tree into disconnected tuples, the model loses sight of the global spatial constraints and long-distance dependencies across complex, multi-line equations, resulting in structural false positives during retrieval.

*Transformer-Based Dense Retrieval.* Driven by the success of Sentence-BERT [**?** ] and ColBERT [**?** ] in text retrieval, recent MathIR architectures have attempted to adapt deep transformer models to mathematical notation (e.g., MathBERT [**?** ]). These models convert formulas into a one-dimensional sequence of LaTeX or MathML tokens and use multi-head self-attention to learn how the symbols relate to each other in context. The goal is to project formulas into the same dense vector space as natural language, allowing for seamless semantic clustering (e.g., drawing $E = mc^2$ close to the text token "mass-energy equivalence").

Limitations: Adapting NLP transformers to handle strict mathematical contexts creates significant architectural limitations. First, standard Byte-Pair Encoding (BPE) tokenizers frequently destroy the logical boundaries of LaTeX commands, fragmenting operators into meaningless sub-word tokens. Second, 1D self-attention mechanisms, which rely on positional encodings, struggle to accurately model the 2D spatial arrangements (fractions, matrices, limits) inherent to mathematics. Finally, the quadratic computational complexity $O(n^2)$ of self-attention across long, serialized MathML sequences creates major latency overheads. This causes a serious bottleneck that limits the retrievers in a massive dataset like ARQMath.

*The Case for Graph Neural Networks.* Our MathRAG system bridges the gap between these paradigms. Rather than flattening the topology into N-grams (like Tangent-CFT) or forcing 2D spatial math into 1D sequence transformers (like MathBERT), we represent the explicit SLT and OPT structures natively as Graph Neural Networks (GNNs). By utilizing graph convolutions, we preserve the rigorous structural precision of tree-based methods while achieving the high-speed dense retrieval latency of vector-based models.

*3.2.2 Topological Graph Formulation.* To achieve high-precision mathematical alignment, our MathRAG architecture relies on explicit topological graph modeling. Upon receiving a raw LaTeX query, the system utilizes LaTeXML to perform deterministic, semantic parsing, yielding two distinct tree structures:

- **Symbol Layout Tree (SLT):** Derived from Presentation MathML, the SLT represents the explicit 2D spatial arrangement of the symbols. We define this as a directed graph $\mathcal{G}_{SLT} = (\mathcal{V}_S, \mathcal{E}_S)$, where nodes $v \in \mathcal{V}_S$ represent visual tokens (e.g., identifiers, operators) and edges $e \in \mathcal{E}_S$ represent spatial relationships (e.g., Next, Above, Below, Subscript).

- **Operator Tree (OPT):** Derived from Content MathML, the OPT captures the underlying mathematical execution order. We define this as $\mathcal{G}_{OPT} = (\mathcal{V}_O, \mathcal{E}_O)$, where nodes represent operators or operands, and edges dictate the hierarchical application of operations, strictly independent of visual formatting.

By preserving these structures as explicit graphs rather than flattened sequences, we allow the subsequent neural layers to natively exploit the structural inductive biases of mathematical notation.

*3.2.3 Dual GNN Architecture and Cross-Attention Fusion.* The core formula embedding engine is a Dual Graph Neural Network (GNN) implemented via PyTorch Geometric [**?** ]. The pipeline processes the visual and semantic topologies in parallel before fusing them.

*Graph Convolutional Ingestion.* Both $\mathcal{G}_{SLT}$ and $\mathcal{G}_{OPT}$ are passed through stacked Graph Convolutional Network (GCN) layers. For a given node $v_i$ at layer $l$, the message-passing update rule is defined as:

$$h_i^{(l+1)} = \sigma\left( \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{c_{i,j}} W^{(l)} h_j^{(l)} \right) \tag{2}$$

where $\mathcal{N}(i)$ is the localized neighborhood of $v_i$, $c_{i,j}$ is a normalization constant based on node degrees, $W^{(l)}$ is the layer-specific learnable weight matrix, and $\sigma$ is a non-linear activation function (ReLU). This allows the network to capture both localized symbol neighborhoods and hierarchical tree depth. We denote the final graph-level embeddings obtained via global mean pooling as $H_{SLT} \in \mathbb{R}^d$ and $H_{OPT} \in \mathbb{R}^d$.

*Multi-Head Cross-Attention Layer.* To synthesize the spatial layout with its semantic mathematical execution, $H_{SLT}$ and $H_{OPT}$ are routed through a multi-head cross-attention mechanism. We treat the semantic tree as the primary query, allowing it to attend to the visual layout:

$$Q = H_{OPT} W_Q, \quad K = H_{SLT} W_K, \quad V = H_{SLT} W_V \tag{3}$$

The attention weights are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left( \frac{QK^\top}{\sqrt{d_k}} \right) V \tag{4}$$

This fusion allows the model to correctly align structural layouts with their corresponding mathematical operators. The resulting tensor is passed through a Multi-Layer Perceptron (MLP) projection head and subjected to $L_2$ Normalization to produce the final 256-dimensional dense query vector $v_q \in \mathbb{R}^{256}$.

*Queries with Multiple Formulas.* If a user query contains multiple formulas, let the extracted set be $\Phi(q) = \{\phi_1, \ldots, \phi_m\}$. The system encodes each formula independently, producing one dense query vector per formula:

$$\mathbf{v}_{q,i} = f_{\text{gnn}}(\phi_i), \quad i \in \{1, \ldots, m\} \tag{5}$$

Each $\phi_i$ retrieves its own ranked candidate set. The aggregated candidate pool is the union of those sets,

$$C(q) = \bigcup_{i=1}^{m} C(\phi_i) \tag{6}$$

and a document-level score can be defined by the strongest matching query formula,

$$S_{\text{multi}}(q, d) = \max_{1 \le i \le m} S(\phi_i, d) \tag{7}$$

so that a document is retained if it is strongly aligned with at least one formula appearing in the original query.

*3.2.4 Two-Stage Retrieval Pipeline.* To balance latency with precision across an 8.3 million document dataset, retrieval is executed in two distinct phases: high-recall candidate generation followed by structural reranking.

*Stage 1: Alpha-Weighted Hybrid Search.* The initial candidate pool is generated by fusing sparse and dense retrieval metrics. The dense query vector $v_q$ is evaluated against the document vectors $v_d$ in an inverted FAISS index using exact inner-product similarity: $S_{\text{dense}} = v_q \cdot v_d$. Simultaneously, a BM25 inverted index calculates a sparse lexical score $S_{\text{sparse}}$ based on exact-match keyword frequencies. The scores are Min-Max normalized and fused using an $\alpha$-weighting parameter (empirically set to emphasize structural density):

$$S_{\text{hybrid}} = \alpha \hat{S}_{\text{dense}} + (1 - \alpha) \hat{S}_{\text{sparse}} \tag{8}$$

This yields the top-$K$ candidates for the reranking phase.

*Stage 2: Structural Subtree Reranking.* Inspired by the precision mechanisms of Tangent-S [**?** ], the top-$K$ candidates undergo deterministic structural verification. The system shreds the query's OPT and SLT into structural paths up to a maximum depth of 4. An Inverse Document Frequency (IDF) weighted subtree similarity score is calculated, heavily penalizing candidates that share superficial symbols but lack exact path overlaps. This deterministic reranker guarantees that commutative operator alignments and exact bracket closures dominate the top ranks.

*3.2.5 System Optimization and Evaluation.* To maintain low latency during Stage 2 reranking, we bypassed the severe disk I/O bottlenecks inherent to compressed Parquet file lookups. We migrated the raw MathML strings of the 8.3 million dataset into a heavily indexed SQLite caching layer, reducing XML retrieval times to sub-millisecond ranges.

*ARQMath-3 Metrics.* The pipeline was evaluated on the ARQMath-3 Task 2 benchmark using the formal "Prime" configuration. To establish a rigorous, apples-to-apples baseline for mathematical vector spaces, we independently re-evaluated the Tangent-CFT [**?** ] embeddings across its semantic (OPT), visual (SLT), and Early Fusion modalities without the aid of external competition ensembling.

As detailed in Table 1, naive structural N-gram extraction struggles to capture holistic mathematical meaning. For instance, the naive fusion of pure semantics with visual layouts dilutes the underlying mathematical intent. While the Fused Tangent-CFT model establishes a respectable baseline with an nDCG@5 of 0.6188, it cannot account for complex topological dependencies across larger equations.

In contrast, our MathRAG Dual GNN architecture, which explicitly preserves topological dependencies and utilizes cross-attention fusion, achieves a massive leap in precision. The proposed pipeline yields an nDCG@5 of 0.7753 and an nDCG@10 of 0.7182. This

**Table 1: Performance Comparison of TangentCFT Baselines and Proposed Graph Neural Network on ARQMath-3 Task 2 (Prime Metrics)**

| Metric | TangentCFT (SLT) | TangentCFT (OPT) | TangentCFT (Fused) | Proposed GNN |
|---|---|---|---|---|
| bpref | 0.3752 | 0.4362 | 0.4487 | 0.4189 |
| nDCG@5 | 0.4708 | 0.6168 | 0.6188 | **0.7753** |
| MAP@5 | 0.0953 | 0.1185 | 0.1187 | **0.1570** |
| P@5 | 0.4211 | 0.5737 | 0.5632 | **0.7437** |
| nDCG@10 | 0.3862 | 0.5274 | 0.5381 | **0.7182** |
| MAP@10 | 0.1091 | 0.1512 | 0.1518 | **0.2193** |
| P@10 | 0.2987 | 0.4342 | 0.4329 | **0.6338** |
| nDCG@100 | 0.2862 | 0.3786 | 0.3986 | **0.5398** |
| MAP@100 | 0.1435 | 0.2251 | 0.2330 | **0.4062** |
| P@100 | 0.0829 | 0.1225 | 0.1283 | **0.1851** |
| nDCG@1000 | 0.3815 | 0.4701 | 0.4923 | **0.5372** |
| MAP@1000 | 0.1577 | 0.2410 | 0.2501 | **0.4062** |
| P@1000 | 0.0174 | 0.0206 | 0.0215 | 0.0185 |

+15.6% absolute improvement in top-5 ranking is absolutely critical for the overarching MathRAG system, as injecting hallucinated or structurally incorrect mathematical context into the generative LLM's narrow context window severely degrades final response quality.

## 3.3 Image Search

The image retrieval branch searches a mathematical image dataset derived from Math Stack Exchange, MathOverflow, and Mathematica. This branch is presented through a LongCLIP-based retrieval pipeline that maps both text queries and images into a shared embedding space and returns ranked image matches together with metadata and source URLs.

*3.3.1 Visual Dataset and Dataset.* The image dataset was obtained through scraping tens of thousands of mathematical question and answers online. Each entry stores an `image_id`, source label, title, source URL, and file path. The `image_id` encodes the originating post and image position, which makes it possible to trace a retrieved figure back to the discussion thread from which it was collected. During indexing, the dataset iterator skips images that are missing or unreadable on disk.

*3.3.2 Vision-Language Embedding Model.* The image branch uses LongCLIP to embed both images and text queries into a shared normalized feature space. A fine-tuned checkpoint, trained on these images, is used during production. Image queries are processed by the LongCLIP vision encoder, and text queries are tokenized with `longclip.tokenize(..., truncate=True)` before being encoded into the same retrieval space.

*Image Encoding.* An input image $I$ is preprocessed by the LongCLIP image pipeline and then passed to the vision encoder $f_I$. The resulting vector is $L_2$-normalized:

$$\mathbf{v}_I = \frac{f_I(I)}{\|f_I(I)\|_2} \in \mathbb{R}^d \tag{9}$$

If the image is an SVG, it is rasterized to PNG before encoding.

*Text Encoding.* For a text query $q$, the input is tokenized with `longclip.tokenize(..., truncate=True)` and encoded by the LongCLIP text encoder into the same joint retrieval space:

$$\mathbf{v}_q^T = \frac{g(q)}{\|g(q)\|_2} \in \mathbb{R}^d \tag{10}$$

The shared space allows direct comparison between text queries and image embeddings.

*3.3.3 Offline Index Construction.* Each image in the dataset is encoded once and stored in a FAISS `IndexFlatIP` index. If the first successfully encoded image has dimension $d$, the resulting matrix of stored embeddings is $\mathbf{V} \in \mathbb{R}^{N \times d}$. A parallel JSON metadata file preserves insertion order so that each FAISS row can be mapped back to the corresponding source image.

*3.3.4 Online Retrieval.* At service level, the image branch supports both text-to-image and image-to-image retrieval. If the user input resolves to a valid file path, it is encoded as an image; otherwise it is encoded as text. Because all vectors are $L_2$-normalized, inner-product search in FAISS is equivalent to cosine similarity:

$$S(q, i) = \hat{\mathbf{v}}_q \cdot \mathbf{v}_{I_i} \tag{11}$$

The service returns the top-$K$ ranked images together with score, title, source, URL, and file path. In the full RAG pipeline, the orchestrator then scrapes the associated source pages and forwards the resulting text as evidence to the language model.

## 3.4 Answer Generation

The final stage of MathRAG is evidence normalization and answer generation. Although the retrieval system is multimodal, the generator is a local text-only language model. The orchestrator therefore converts the output of each retrieval branch into text before generation.

Text retrieval contributes scraped passages or stored document bodies. Formula retrieval contributes the text of posts associated with formula matches. Image retrieval contributes the text scraped from posts linked to the retrieved image results. These blocks are inserted into a structured prompt with explicit labels that indicate which branch supplied the evidence. If a modality returns no usable results, its block is simply omitted.

Let $c_{\text{text}}$, $c_{\text{formula}}$, and $c_{\text{image}}$ denote the textualized context blocks returned by the three branches. The final prompt can be written as

$$P(q) = \text{concat}\big(c_{\text{text}}, c_{\text{formula}}, c_{\text{image}}, q\big) \tag{12}$$

Missing modalities are represented as empty strings. The language model then performs conditional generation over the constructed prompt:

$$\hat{a} = g_\theta(P(q)) \tag{13}$$

A post-processing step extracts the final answer from the raw model output.

Answer generation is performed with a HuggingFace `text-generation` pipeline backed by `Llama-3.1-8B-Instruct`. The model runs in half precision on GPU when available and in full precision on CPU otherwise. If a LoRA adapter is present, it is loaded before inference. Generation is deterministic, with sampling disabled, and the maximum output length is controlled by the `RAG_MAX_TOKENS` environment variable.

The raw model output is post-processed to extract a concise final answer. The pipeline first checks for a LaTeX `\boxed{...}` expression and returns the last such match. If no boxed answer is found, it scans backward for lines beginning with answer-oriented preambles such as "the answer is," "result:", or "therefore," and strips the preamble. As a final fallback, it returns the last non-empty line.

## 4 Experimental Results

The experimental evidence is organized around component-level retrieval evaluation rather than a single end-to-end question-answering benchmark. That distinction matters in this system because the orchestrator performs several transformations between retrieval and generation, including scraping, deduplication, lightweight reranking, and prompt assembly. The experimental evidence presented here should therefore be read as evidence for the retrieval components that support the overall pipeline.

### 4.1 Formula Retrieval Results

The clearest component result is the ARQMath-3 Task 2 evaluation reported in Table 1. Across early precision metrics, the proposed graph-based formula retriever outperforms the Tangent-CFT baselines. The improvement is strongest at shallow cutoffs such as nDCG@5, MAP@5, and P@5, which is important for a RAG setting because only a small fraction of retrieved evidence can fit into the final prompt. At the same time, the lower bpref and P@1000 values indicate that the model's largest gains are in high-confidence top-ranked results rather than in deep-recall behavior.

### 4.2 Text Retrieval Results

Text retrieval is also evaluated against ARQMath-3 Task 1. In that study, MathBERTa, SPECTER 2.0, and BGE-M3 were compared as dense retrievers. BGE-M3 produced the strongest reported text results, reaching nDCG@5 = 0.5709, $P$@5 = 0.6590, $P'$@10 = 0.4692, and bpref = 0.2628, while MathBERTa reached $P'$@10 = 0.2026. These results are useful for understanding the behavior of candidate text encoders, but they should be distinguished from the deployed text service used in the live pipeline, which serves a fine-tuned `all-mpnet-base-v2` checkpoint through OpenSearch.

### 4.3 Image Retrieval Results

The image branch is evaluated on the MATVB benchmark, which provides a paired text-to-image retrieval setting for mathematical figures. The benchmark includes two query sets with shared query identifiers: `MATVB_query_nl.json` for short natural-language queries and `MATVB_query_caption.json` for longer caption-style queries. Relevance judgments are provided in TREC format, with graded relevance labels for retrieved images.

In the MATVB evaluation pipeline, every image in the collection is encoded offline, every text query is encoded into the same embedding space, and ranking is performed by cosine similarity. Because both image and text embeddings are $L_2$-normalized, cosine similarity is computed as the inner product between query and image vectors. The script reports four ranking metrics: mean reciprocal rank (MRR), Recall@5, Recall@10, and nDCG@100. This metric set captures both early-hit behavior and deeper ranking quality, which is appropriate for an image branch whose outputs may later be truncated before prompt construction.

Let $q_j$ denote a MATVB text query and let $I_i$ denote an indexed image. Using LongCLIP embeddings, the benchmark ranking score is

$$S_{\text{MATVB}}(q_j, I_i) = \mathbf{v}_{q_j}^T \cdot \mathbf{v}_{I_i} \tag{14}$$

and the evaluated run for each query is obtained by sorting all candidate images in descending order of $S_{\text{MATVB}}(q_j, I_i)$.

The image evaluation protocol is centered on the LongCLIP-based retriever. This keeps the benchmark protocol aligned with the image model described in the system section and makes the image branch easier to present as one consistent component of the overall pipeline.

### 4.4 Implications for the Full Pipeline

Taken together, the component results support the central design decision of MathRAG: specialized retrievers are valuable because the evidence needed for a mathematical answer does not come from one representation alone. The formula results justify a dedicated symbolic retriever, while the text experiments show that model choice materially affects the quality of narrative context passed to the generator. The image branch extends this idea to visual evidence, and a task-specific end-to-end answer benchmark would quantify its effect on final response quality.

## 5 Discussion

The main strength of MathRAG is architectural clarity. Each modality is retrieved with a method that matches its structure, and the orchestration layer keeps the branches loosely coupled. This makes the system easier to debug, replace, and scale than a design that attempts to force text, formulas, and images into one retrieval representation from the start.

The system also exposes an important tradeoff. Retrieval is genuinely multimodal, but generation is still text-only. In practice, that means the system benefits from image and formula retrieval mainly through textual evidence scraped from the posts linked to those results. This keeps the generator simple and local, but it also means that some modality-specific information is compressed before it reaches the language model. Direct formula or vision-aware generation would test whether preserving more modality-specific structure improves answer quality beyond this text-normalized design.

## 6 Conclusion

MathRAG is a modular multimodal RAG system for mathematical question answering. Its central design choice is to keep retrieval specialized by modality while normalizing evidence into a single prompt for generation. The text branch, formula branch, and image branch each target a different form of mathematical evidence, and the orchestrator combines them into one answering pipeline. The result is a practical architecture for mathematical QA that is grounded in the representations already present in real technical discussions.

## References

Received TBD; revised TBD; accepted TBD